

Red-Teaming Prompts: Finding Failures in AI Systems

Red-Teaming Prompts: Finding Failures in AI Systems

A Research Report — July 2026

1. Executive Summary

AI red-teaming is the practice of adversarially testing AI and LLM systems to surface safety and security failures before real attackers or ordinary users find them. It differs from traditional application security testing in a fundamental way: classic pentesting checks code, infrastructure, and API logic, while AI red-teaming probes model *behavior* under adversarial pressure — a surface that static analysis and conventional scanners were never designed to cover.

Key takeaways from current research:

- **Human, multi-turn red-teaming still outperforms automation by a wide margin.** Automated single-shot attacks report low attack success rates that understate real risk; multi-turn human red-teaming can expose failures at rates up to 75%.
 - **No model is broadly robust.** A large benchmark study testing 18 attack types against 33 models found that no single model defended against the full range of attacks, and simply scaling a model family up does not reliably improve robustness.
 - **The threat surface has moved from single prompts to agents.** In 2026, tool-calling agents and Model Context Protocol (MCP) servers introduce failure modes — privilege escalation across chained tool calls, cross-hop data leakage — that single-prompt scanners were never built to catch.
 - **Tooling has matured into a small standard set** (Garak, Promptfoo, PyRIT, DeepTeam, PromptBench), increasingly mapped to governance frameworks like the OWASP Top 10 for LLMs, OWASP ASI, NIST AI RMF, and MITRE ATLAS.
 - **The gap between incidents and evaluation is widening.** Reported AI-related incidents rose sharply year-over-year while formal responsible-AI evaluation remains comparatively rare, which is part of what's driving investment in structured red-teaming programs and regulation such as the EU AI Act.
-

2. What Red-Teaming Is and Why It's Different

AI red-teaming tests three distinct layers of a deployed system:

1. **The model itself** — its trained behavior, safety alignment, and refusal patterns.
2. **The application wrapped around it** — the system prompt, retrieval pipeline, and business logic.
3. **The agentic workflow** — tools the model can call, multi-step plans, and other AI components it coordinates with.

Because language models are non-deterministic and instruction-driven rather than code-driven, most attacks happen at the prompt and conversation level rather than the code level. A red-teaming exercise typically involves throwing hostile inputs at a system to find where it breaks, leaks data, or does something its developers never intended — with the same intent a real attacker would bring.

What counts as a “failure” is defined by the deployer’s threat model. For a frontier AI lab, the harm categories of greatest concern typically include weapons uplift (chemical, biological, radiological, nuclear) and other catastrophic misuse categories. For a typical enterprise deploying an LLM-powered product, the list is usually narrower and shaped by whatever legal, security, and product teams are accountable for — things like data leakage, off-brand or offensive outputs, or the system being manipulated into unauthorized actions.

3. Taxonomy of Prompt-Based Attack Techniques

Researchers generally divide LLM attacks along two axes: the attacker’s level of access, and the mechanism of manipulation.

3.1 Access-based split

- **Black-box attacks** — the attacker interacts only through the model’s normal input/output interface, with no visibility into weights or training. This includes prompt modification, heuristic and reinforcement-learning-based prompt optimization, LLM-assisted prompt generation, and multi-turn manipulation. This category covers essentially all “prompt red-teaming.”
- **White-box attacks** — the attacker has access to model internals (gradients, weights, architecture), enabling techniques like

gradient-based adversarial prompt optimization. This is a research/lab concern more than a deployment-testing one.

3.2 Mechanism-based categories

Category	Description	Representative techniques
Role hijacking / persona override	Convincing the model to adopt an alternate persona that ignores its normal behavioral constraints	Fictional “unlocked” personas, authority role-play
Instruction hijacking	Embedding new instructions that override the original system/user intent	Direct override phrasing, delimiter confusion
Context manipulation	Reframing the conversation’s context so a harmful request appears benign	Scenario camouflage, nested fictional framing
Obfuscation / encoding	Disguising harmful content so pattern-based filters miss it	Ciphers, unusual formatting, translation, token-level tricks
Multi-turn escalation	Building rapport or context over several turns before introducing the harmful ask	Gradual trust-building, incremental requests
Social engineering / persuasion	Using human-style persuasion tactics — empathy, authority claims, emotional appeals — to override refusal behavior	Persuasion-based jailbreak frameworks
Framing tricks	Presenting a harmful request as educational, academic, or research-oriented to make refusal seem unreasonable	“Educational framing,” “research context” categories in large jailbreak taxonomies
Nesting / nested nesting	Embedding a harmful query inside layers of an innocent story or scenario	Approaches described as a “wolf in sheep’s clothing,” which fool models by burying intent under benign-seeming wrapping
Disguise and reconstruction	Splitting a harmful request into disguised sub-components that the model itself reconstructs through guided questioning	Frameworks that rebuild the harmful whole from innocuous-looking parts
Multi-modal jailbreaking	Smuggling adversarial intent through images, audio, or video rather than text alone	Cross-modal fusion attacks

3.3 Scale of the problem

Empirical studies illustrate just how effective these techniques can be at scale:

- One large-scale collection aggregated **1,405 jailbreak prompts across 131 online forums**, finding success rates as high as 95% for some prompt patterns.
- A newer nested-prompt method reportedly improved jailbreak performance by roughly 40% over earlier approaches.
- A tree-search framework pairing an “attacker” model against an “evaluator” model achieved jailbreaks on more than 80% of prompts against GPT-4-Turbo and GPT-4o using only black-box access, while using notably fewer queries than earlier search-based methods.
- A comprehensive benchmark testing 18 distinct attacks against 33 different LLMs found that **no single model successfully defended against the full range of attack types**, and that increasing model size within a family (7B → 70B parameters) did not guarantee improved robustness.

4. The Human-vs-Automation Gap

Perhaps the most consequential finding in current red-teaming research is the gap between automated and human testing. Single-turn automated attack algorithms tend to produce reassuringly low attack success rates. Multi-turn human red-teaming, by contrast, has been shown to expose failures at rates up to roughly 75%.

This has two practical implications:

1. **Static benchmark scores understate real deployment risk.** A model that scores well against an automated jailbreak suite can still be broken relatively easily by a motivated, creative human across a multi-turn conversation.
2. **Human expertise remains essential, not optional.** A tester who stays current with the latest published techniques will typically find issues that a three-month-old automated test suite cannot, because new attack techniques emerge on a near-weekly basis.

5. Methodology: How Structured Red-Teaming Programs Operate

A widely used practitioner framework structures a red-teaming engagement into five phases:

1. **Reconnaissance** — understanding the target system’s system prompt, tools, data sources, and business logic.
2. **Attack generation** — producing candidate adversarial prompts, drawing on both known taxonomies and novel creative approaches.
3. **Execution** — running the attacks against the live system (chat interface, API, or agent workflow).
4. **Validation** — confirming which attempts actually produced a genuine policy violation or unintended behavior, as opposed to false positives.
5. **Mitigation, with re-test** — feeding findings back into model training, guardrails, or deployment policy, then re-running the same attacks to confirm the fix holds.

Two further principles show up consistently across sources:

- **Model-vs-system distinction.** It matters whether a failure originates in the base model’s training/alignment, or in the application layer wrapped around it (system prompt design, retrieval pipeline, tool permissions). The fix differs substantially depending on which layer is responsible.
- **Continuous discipline, not a one-time gate.** Red-teaming works best as an ongoing engineering practice rather than a pre-launch checkbox. Recommended cadence: before initial production deployment, after any model upgrade or system-prompt change, and on a recurring monthly-or-quarterly schedule to catch newly emerging attack techniques. Off-the-shelf, generic test suites are described as insufficient on their own — custom test suites targeting a system’s specific configuration are considered essential.

6. The 2026 Shift: From Chat Prompts to Agentic Systems

Historically, “red-teaming an LLM” meant testing a single conversational endpoint with adversarial text. That threat model has expanded considerably. Modern LLM deployments increasingly involve:

- **Tool-calling agents** that can take real-world actions (searching, emailing, executing code, calling APIs).
- **Autonomous multi-step workflows** where a single compromised step can cascade.
- **Model Context Protocol (MCP) servers** connecting models to external systems and data sources.

These introduce failure modes that single-prompt scanners simply don’t cover — for example, an agent being tricked into invoking the wrong tool, escalating privileges through a chain of tool calls, or leaking sensitive data as information passes between tool “hops.” These failure classes map onto specific categories in the OWASP framework for LLM applications, notably **Excessive Agency** and **Sensitive Information Disclosure**.

7. OWASP Top 10 for LLM Applications (2025 Edition) — Detailed

The OWASP Top 10 for LLM Applications is a community-built, consensus classification of the most critical security risks in LLM-based applications, maintained by the OWASP GenAI Security Project. The first edition shipped in 2023; the current 2025 edition reorganized and expanded the list to reflect the shift from standalone chatbot deployments toward production agentic systems with tool access, retrieval-augmented generation (RAG) pipelines, and multi-model architectures. It is not a compliance framework in itself, but it is increasingly treated by regulators and auditors as the baseline for what a reasonable AI security program should cover, and demonstrating coverage against it is cited as supporting evidence for NIST AI RMF’s adversarial-testing requirements and for EU AI Act cybersecurity obligations on general-purpose models with systemic risk.

The ten categories:

- **LLM01 — Prompt Injection.** Crafted input causes the model to follow an attacker’s instructions instead of the developer’s, overriding system-prompt constraints without authorization. This is described as the most actively exploited LLM vulnerability in production, especially the *indirect* variant, where adversarial instructions are hidden inside external content the model retrieves (a document, web page, or email) rather than typed directly by the user. Indirect injection is particularly dangerous because it requires no special access to the application and has already shown up in real-world incidents against major AI products.
- **LLM02 — Sensitive Information Disclosure.** Failure to protect against disclosure of sensitive information in LLM outputs, which can create legal exposure or erode competitive advantage. Common in early-stage deployments where credentials or proprietary logic end up embedded in prompts or context.
- **LLM03 — Supply Chain.** Third-party models, datasets, packages, and plugins introduce vulnerable or malicious components — for instance, a model checkpoint downloaded from a public hub that executes remote code on load through unsafe deserialization, or a library dependency with a known vulnerability.
- **LLM04 — Data and Model Poisoning.** Manipulated training, fine-tuning, or embedding data introduces backdoors, bias, or degraded behavior. If an attacker poisons underlying data (e.g., uploading a manipulated document into a knowledge base), the model will retrieve and treat it as ground truth.
- **LLM05 — Improper Output Handling.** Model output is passed to downstream systems without validation or sanitization, enabling injection attacks, server-side request forgery, or even remote code execution — the AI-era equivalent of failing to sanitize SQL inputs.
- **LLM06 — Excessive Agency.** The system grants the model too much functionality, too many permissions, or too much autonomy, so a manipulated model can take harmful actions. This category is typically broken into three subcomponents: *excessive permissions*

(tools that exceed the task’s actual scope), *excessive functionality* (high-impact tools like shell execution or production database writes), and *excessive autonomy* (consequential actions taken with no human confirmation gate). It’s the single most expanded entry in the 2025 update, reflecting the rise of tool-calling agents, and it’s also where “tool composition” risk lives — two individually-reviewed and seemingly safe tools (e.g., a file-reader and an email-sender) can combine into a data-exfiltration path that no single review caught.

- **LLM07 — System Prompt Leakage.** The system prompt — including any instructions or secrets it should never have contained in the first place — is exposed to users or attackers, often through direct requests (“what are your instructions?”) or indirect tricks (asking the model to translate or repeat its preceding text). Once an attacker has the system prompt, they can craft far more effective jailbreaks against that specific deployment.
- **LLM08 — Vector and Embedding Weaknesses.** Vulnerabilities in the vector databases and embedding pipelines that power RAG systems — for example, a shared vector database where different customers’ data is separated only at the application layer after retrieval, allowing a crafted query to pull another tenant’s data into context.
- **LLM09 — Misinformation.** The risk that a model produces confidently-stated but false or misleading content, which downstream systems or users treat as authoritative.
- **LLM10 — Unbounded Consumption.** Uncontrolled resource or query usage enables denial-of-service conditions and runaway infrastructure costs — especially dangerous in agent loops that can call expensive tools or APIs repeatedly without a circuit breaker.

8. OWASP Top 10 for Agentic Applications (ASI, 2026 Edition)

As agentic systems — models with memory, tool access, and multi-step autonomous execution — became the dominant deployment pattern, OWASP introduced a companion list specifically for agent-specific risk: the **OWASP Top 10 for Agentic Applications (ASI)**, 2026 edition. It explicitly builds on and extends the LLM Top 10 rather than replacing it: each agentic risk relates to one or more foundational LLM risks but introduces new attack vectors and amplified impact because of autonomy, tool integration, and multi-agent coordination. For example, ASI01 (Agent Goal Hijack) merges prompt injection (LLM01) with excessive autonomy (LLM06) — but because the agent executes multiple steps autonomously, the impact of a successful hijack is amplified well beyond what a single bad response could cause.

Selected categories from the ASI Top 10:

- **ASI01 — Agent Goal Hijack.** An attacker redirects the agent’s overall objective mid-task, rather than just manipulating a single response.
- **ASI02 — Tool Misuse & Exploitation.** The agent is tricked into invoking a tool incorrectly, with the wrong parameters, or for an unintended purpose.
- **ASI03 — Agent Identity & Privilege Abuse.** The agent’s own credentials or identity are abused to escalate privileges or act outside intended scope.
- Further categories address dynamic runtime composition (agents discovering and integrating new tools/components *during* execution, as opposed to LLM03’s static, pre-deployment supply-chain concern), inter-agent trust and communication, persistent memory manipulation, and other entirely new vulnerability classes that don’t have a direct analogue in single-model LLM applications.

A related, more granular taxonomy specifically for Model Context Protocol (MCP) systems — sometimes referenced as “MCP-38” — outlines 38 distinct risk categories spanning authentication bypass, privilege escalation, and adversarial tool-description manipulation, where a compromised MCP server sends misleading tool descriptions that push the model toward invoking capabilities its operator never intended it to have access to. Given the operational significance of this risk class, government bodies have begun issuing formal guidance on MCP security specifically, treating it as a live infrastructure concern rather than a theoretical one.

9. Defense-in-Depth: Mitigation Strategies by Risk Category

Because prompt-level filtering alone cannot catch every variant of every technique above, current guidance converges on a layered, “defense-in-depth” architecture rather than any single control. Representative mitigations, mapped to the risk they address:

- **Treat the model as untrusted output, not as a trusted backend.** Validate and sanitize every model output before it reaches a downstream system, the same way you would treat any other untrusted input — this addresses improper output handling (LLM05) and prevents generated URLs or code from being executed or fetched blindly (which can otherwise lead to server-side request forgery).
- **Complete mediation in downstream systems.** Authorization checks belong in the downstream system, not in the model’s own judgment — never let the model decide access control; validate every request against real security policy.
- **Separate sensitive data from system prompts entirely.** Credentials, API keys, and permission structures should never be embedded in a system prompt in the first place, since prompt leakage (LLM07) should be assumed to be a matter of when, not if.
- **Zero-trust between agents.** In multi-agent systems, agents should not implicitly trust each other; techniques like mutual authentication between agents and cryptographically binding tool calls to a signed statement of the original user’s intent help prevent one compromised or manipulated agent from cascading into others.
- **Circuit breakers and rate limits.** Strict cost ceilings and rate limits stop a looping or manipulated agent from causing runaway resource consumption (LLM10) before it becomes a major incident.
- **Hard tenant isolation in vector databases.** Cryptographic namespace separation — not just application-layer filtering — prevents

cross-tenant data leakage through embedding retrieval (LLM08).

- **Cumulative, cross-turn safety evaluation.** Because multi-turn escalation attacks distribute adversarial intent across several messages, defenses need to track intent cumulatively across a conversation, not just evaluate each message in isolation.
- **Input normalization before filtering.** Since obfuscation and encoding attacks work by hiding intent from pattern-matching filters, inputs should be normalized and common encodings decoded *before* safety evaluation runs, not after.
- **Reasoning-chain review, not just final-output review.** For models that expose intermediate reasoning, that reasoning should itself be evaluated for adversarial logic, since a benign-looking final answer can still be reached via a manipulated chain of thought.
- **No single layer is sufficient.** Every source surveyed converges on the same point: pattern-based detection alone will always be evaded by sufficiently novel framing, so architectural controls — privilege separation, continuous monitoring, and a limited “blast radius” for any single component — matter as much as the safety training of the model itself.

10. Tooling Landscape (2026)

The most frequently cited open-source frameworks for LLM red-teaming in 2026 are:

- **Garak**
- **Promptfoo**
- **PyRIT**
- **DeepTeam**
- **PromptBench** (cited in some practitioner guides alongside the above four)

These frameworks generally probe for prompt injection, jailbreaks, data leakage, and other adversarial failure modes before an application reaches production. Commercial and managed alternatives also exist for organizations that want continuous, human-validated coverage rather than a one-off automated scan — these tend to pair automated agentic testing with human security researchers, since (per Section 4) automation alone tends to underestimate real risk.

Governance and reference frameworks

Findings from red-teaming exercises are increasingly mapped to standardized frameworks, which also serve compliance and procurement purposes:

- **OWASP Top 10 for LLM Applications** — a widely adopted baseline vulnerability list for LLM-specific risks.
- **OWASP ASI 2026** — classifies agent-specific vulnerabilities across 10 distinct risk categories, addressing the agentic shift described in Section 6.
- **NIST AI RMF (AI Risk Management Framework)** — a governance-oriented methodology rather than a technical checklist, increasingly used to anchor enterprise procurement requirements.
- **MITRE ATLAS** — catalogs adversarial techniques against AI systems, with ongoing additions of AI-specific attack methods.

Together, these three types of framework (technical vulnerability list, agent-specific taxonomy, and governance methodology) are described as complementary — useful “used together for comprehensive mapping of adversarial tests to regulatory controls and compliance evidence.”

11. Broader Context: Incidents, Regulation, and the Evaluation Gap

Stanford HAI’s 2025 AI Index reports that AI-related incidents rose by 56.4% year-over-year, reaching 233 recorded incidents in 2024 — while formal responsible-AI evaluations of deployed systems remain comparatively rare. This mismatch between rising real-world incidents and relatively sparse formal evaluation is cited as a key driver behind growing investment in red-teaming tooling and behind regulatory pressure such as the EU AI Act’s compliance documentation requirements, which are specifically pushing European teams toward more structured, auditable red-teaming programs.

12. Practical Recommendations (Synthesized)

Based on the research above, a reasonable working checklist for anyone standing up a red-teaming program:

1. **Don’t rely on automated single-turn scores alone.** Budget for human, multi-turn adversarial testing — it finds substantially more than automated benchmarks.
2. **Test the system, not just the model.** Include the system prompt, retrieval pipeline, and any tool/agent permissions in scope, not just the base model’s raw behavior.
3. **Build custom test suites.** Generic, off-the-shelf probes catch generic vulnerabilities; your specific configuration needs tailored attacks.
4. **Treat it as continuous, not a launch gate.** Re-test after every model upgrade, system-prompt change, and on a recurring schedule — new attack techniques appear regularly.
5. **If you run agents or tool-calling systems, test for agentic-specific failures** (privilege escalation across tool chains, cross-hop

- data leakage) — not just conversational jailbreaks.
6. **Map findings to a recognized framework** (OWASP LLM Top 10 / ASI, NIST AI RMF, MITRE ATLAS) both for internal prioritization and for compliance documentation.
 7. **Close the loop.** Findings should feed back into training, guardrails, or policy — and then get re-tested to confirm the fix actually holds.

Suggested testing cadence

Trigger	Scope	Why
Before initial production launch	Full taxonomy + agentic risks if applicable	Establish a baseline before real users are exposed
After any model upgrade	Full re-test of previously found issues + spot checks on new categories	Model behavior can shift in ways that reopen fixed issues or introduce new ones
After any system-prompt or tool-permission change	Targeted testing of the changed component	Small configuration changes are a common source of regressions
Monthly or quarterly (minimum)	Rotating coverage across the full taxonomy	New attack techniques and jailbreak patterns publish on a near-weekly basis
Continuously, in production	Automated monitoring for known attack signatures	Static point-in-time testing cannot catch drift or novel real-world attacks

13. Glossary

- **Adversarial prompt** — an input specifically crafted to make a model behave in a way its designers didn’t intend.
- **Attack Success Rate (ASR)** — the percentage of adversarial attempts that successfully produce the intended failure; the standard metric for comparing attack techniques and model robustness.
- **Black-box attack** — an attack conducted using only the model’s normal input/output interface, without access to its internal weights or training data.
- **White-box attack** — an attack that uses access to model internals (gradients, weights, architecture) to craft more effective adversarial inputs.
- **Jailbreak** — a prompt or sequence of prompts designed to bypass a model’s safety alignment through conversational input alone, exploiting learned behavioral patterns rather than a code-level vulnerability.
- **Prompt injection** — manipulating a model’s behavior by embedding adversarial instructions in its input, causing it to follow the attacker’s instructions instead of (or in addition to) the developer’s; considered the LLM analogue of command injection in traditional software.
- **Indirect prompt injection** — a variant where the adversarial instructions are embedded not in the user’s direct message but in external content the model retrieves, such as a document, webpage, or email.
- **Excessive agency** — a system design flaw where a model is granted more functionality, permission, or autonomy than its task requires, allowing a manipulated model to take harmful real-world actions.
- **Multi-turn attack** — an attack that unfolds across several conversational exchanges, gradually building context, trust, or incremental commitments rather than attempting the harmful ask in a single message.
- **Red team / blue team** — terminology borrowed from military and cybersecurity exercises; the red team plays the adversary attempting to break a system, while the blue team plays the defender.
- **Model Context Protocol (MCP)** — a protocol connecting models to external tools, data sources, and services; a significant and rapidly growing attack surface in 2026 agentic deployments.
- **Defense-in-depth** — a security architecture principle in which multiple independent layers of protection are used together, so that no single point of failure compromises the whole system.
- **RAG (Retrieval-Augmented Generation)** — an architecture where a model’s responses are grounded in content retrieved from an external knowledge base or document store at query time; a common vector for indirect prompt injection and data poisoning.

Sources Consulted

- Synack — *Best AI Red Teaming Tools to Find AI Security Vulnerabilities in 2026*
- AppSecSanta — *LLM Red Teaming Guide 2026: Tools, Attacks & Methodology*
- Confident AI — *5 Best AI Red Teaming Tools to Find AI Security Vulnerabilities in 2026*
- Galileo — *8 Red Teaming Strategies for LLMs and Agents*

- Kili Technology — *LLM Red Teaming in 2026: How Frontier Labs Test AI*
- DataVLab — *Red-Teaming LLMs 2026: A Practitioner's Guide*
- SnailSploit — *LLM Jailbreak Techniques: A Technical Taxonomy*
- arXiv 2505.04806 — *Red Teaming the Mind of the Machine: A Systematic Evaluation of Prompt Injection and Jailbreak Vulnerabilities in LLMs*
- arXiv 2510.15476 — *SoK: Taxonomy and Evaluation of Prompt Security in Large Language Models*
- arXiv 2511.02376 — *AutoAdv: Automated Adversarial Prompting for Multi-Turn Jailbreaking of Large Language Models*
- arXiv 2507.04673 — *Trojan Horse Prompting: Jailbreaking Conversational Multimodal Models by Forging Assistant Message*
- arXiv 2605.26409 — *Jailbreak susceptibility prediction and mitigation via the behavioral geometry of models*
- arXiv 2510.13893 — *Guarding the Guardrails: A Taxonomy-Driven Approach to Jailbreak Detection*
- EmergentMind — *Jailbreak Prompt Engineering*
- Brian D. Colwell — *Defining The Prompt-Level AI Jailbreaking Techniques*
- arXiv 2605.15118 — *Talk is (Not) Cheap: A Taxonomy and Benchmark Coverage Audit for LLM Attacks*
- OWASP Foundation — *OWASP Top 10 for Large Language Model Applications*
- OWASP GenAI Security Project — *LLMRisks Archive*
- Siemba — *OWASP Top 10 for LLMs (2026) Security Testing & Mitigation Guide for AI Applications*
- DeepTeam — *OWASP Top 10 for Agents 2026*
- Wiz — *OWASP LLM Top 10: A Practitioner's Guide to LLM Security Risks*
- Repello AI — *OWASP LLM Top 10 (2026): The 10 Critical LLM Security Risks Explained*
- Kodem — *OWASP Top 10 for LLM Applications (2025)*
- Alex Ewerlöf — *OWASP Top 10 Agents & AI Vulnerabilities (2026 Cheat Sheet)*